

EventFinder - PHP Files Documentation

This document provides a comprehensive explanation of all PHP files in the EventFinder project, their purposes, and their roles in the application.

📄 Table of Contents

1. [Root Level PHP Files](#)
 2. [Admin PHP Files](#)
 3. [Student PHP Files](#)
 4. [Database Connection](#)
 5. [PHPMailer Integration](#)
-

Root Level PHP Files

1. login.php (`src/php/login.php`)

Purpose: User Authentication and Session Management

What it does:

- Receives login credentials (email and password) from the login form
- Validates user credentials against the database
- Creates a PHP session with user information if credentials are correct
- Stores user details in `$_SESSION`: `user_id`, `user_name`, `user_email`, `user_role`, `phone`, `address`, `photo`
- Redirects users to appropriate homepage based on role (admin → `admin_homepage.html`, student → `student_homepage.html`)
- Returns error if credentials are invalid

Key Process:

1. Gets POST data (email, password)
2. Queries users table with prepared statement
3. Compares password directly with stored password
4. Creates session array on successful match
5. Redirects based on user role

Related Files:

- Frontend: `login.html`
 - Database: `users` table
-

2. signup.php (`src/php/signup.php`)

Purpose: User Registration and Account Creation

What it does:

- Accepts user registration data: `fullname`, `email`, `password`, `phone`, `address`, `profile photo`
- Validates that email doesn't already exist
- Handles profile photo upload with MIME type checking
- Stores uploaded photo in `uploads/profile_photos/` directory
- Generates unique filenames using timestamp + random hex
- Creates new user account in database with "student" role by default
- Automatically logs in the new user and creates session
- Stores photo path in `localStorage` and `sessionStorage`

Key Features:

- File upload validation: Only JPG, PNG, GIF, WebP allowed
- File size limit: 5MB maximum
- Creates upload directory if not exists
- Checks for duplicate emails before registration
- Auto-login after successful signup

Related Files:

- Frontend: `signup.html`
 - Database: `users` table
 - Upload directory: `uploads/profile_photos/`
-

3. get_current_user.php (`src/php/get_current_user.php`)

Purpose: Retrieve Current Logged-In User Information (AJAX Endpoint)

What it does:

- Returns JSON data of currently logged-in user
- Extracts data from `$_SESSION` array
- Used by JavaScript to populate user information in navbar/dashboard
- Includes: user_id, user_name, user_email, user_photo, user_role, phone, address

Response Format:

```
{
  "user_id": 1,
  "user_name": "John Doe",
  "user_email": "john@example.com",
  "user_photo": "uploads/profile_photos/user_123.jpg",
  "user_role": "student",
  "phone": "9812345678",
  "address": "Kathmandu"
}
```

Used by: Student/Admin dashboards for dynamic profile display

4. `get_event_details.php` (`src/php/get_event_details.php`)

Purpose: Fetch Single Event Details (AJAX Endpoint)

What it does:

- Retrieves complete event information by event_id
- Accepts `event_id` from GET request
- Returns event as JSON object with all fields: ename, category, date, time, venue, description, volunteers_needed, photo, created_by
- Requires user to be logged in (checks session)

Response Example:

```
{
  "event_id": 5,
  "ename": "Sports Day 2024",
  "category": "sports",
  "date": "2024-05-15",
  "time": "10:00:00",
  "venue": "College Ground",
  "description": "Annual sports day for all students",
  "volunteers_needed": 10,
  "photo": "uploads/event_photos/event_123.jpg",
  "created_by": 2
}
```

Used by: Event details modal in homepage and event discovery pages

5. `get_upcoming_events.php` (`src/php/get_upcoming_events.php`)

Purpose: Fetch Future/Upcoming Events (AJAX Endpoint)

What it does:

- Queries events table for all events with date $\geq$ today's date
- Sorts by date (ascending order - nearest first)
- Returns maximum 20 events as JSON array
- Used for homepage event carousel showing upcoming events

Query Logic:

- WHERE date $\geq$ CURDATE()
- ORDER BY date ASC
- LIMIT 20

Response: Array of event objects

6. `get_recent_events.php` (`src/php/get_recent_events.php`)

Purpose: Fetch Completed/Past Events (AJAX Endpoint)

What it does:

- Retrieves events that have already occurred (date < today)
- Returns maximum 4 most recent events (sorted newest first)
- Used for "Recent Events" section on homepage

Query Logic:

- WHERE date < CURDATE()
 - ORDER BY date DESC
 - LIMIT 4
-

7. get_popular_events.php (src/php/get_popular_events.php)

Purpose: Fetch Popular/Trending Events (AJAX Endpoint)

What it does:

- Returns past events sorted by number of participants
- Queries events joined with participators count
- Limits to 10 most popular events
- Used for "Popular Events" section on homepage

Query Logic:

- Joins events with participators table
 - Counts participants for each event
 - Filters past events only (date < today)
 - ORDER BY participant count DESC
 - LIMIT 10
-

8. update_user.php (src/php/update_user.php)

Purpose: Update User Profile Information (AJAX Endpoint)

What it does:

- Updates user profile: name, phone, address, email, role (admin), password, photo
- Accepts POST request with user_id and any fields to update
- Handles profile photo upload if provided
- Uses dynamic SQL building to update only provided fields
- Returns success/failure JSON response

Updateable Fields:

- `uname` - Full name
- `phone` - Phone number
- `address` - Home address
- `email` - Email (admin can change)
- `role` - Role assignment (admin only)
- `password` - Password (stored as plain text)
- `photo` - Profile picture

Related Files:

- Used by: `edit_profile.html` (student), `edit_user.html` (admin)
 - Upload directory: `uploads/profile_photos/`
-

9. mail.php (src/php/mail.php)

Purpose: General Email Functionality

What it does:

- Handles general email sending operations
- May be used for notifications, confirmations, or general communications
- Implementation details depend on email configuration

Note: Used as a utility function for various email operations throughout the system

10. auto_mail.php (src/php/auto_mail.php)

Purpose: Automated Email Notifications

What it does:

- Sends automatic emails for system events
- Triggers include: event registration, complaint response, event approval/rejection

- May handle background email processing

Used for:

- Event registration confirmations
- Admin reply notifications to complaints
- Event request approval/rejection notifications

Admin PHP Files

Located in `src/php/admin/` - All require admin role verification

1. `add_event.php` (`src/php/admin/add_event.php`)

Purpose: Create New Event (Admin Only)

Input Parameters:

- `eventName` - Event title
- `eventCategory` - Category (sports, academic, cultural, social, technical, workshop, seminar, music, other)
- `eventLocation` - Venue/location
- `eventDescription` - Detailed description
- `eventDate` - Date in YYYY-MM-DD format
- `eventTime` - Time in HH:MM format (defaults to 10:00:00 if not provided)
- `volunteersNeeded` - Number of volunteers required (integer)
- `eventPhoto` - Image file upload

Process:

1. Validates POST request
2. Checks file type (JPEG, PNG, GIF, WebP only)
3. Generates unique filename with timestamp + random hex
4. Uploads to `uploads/event_photos/`
5. Inserts into events table with `created_by` = admin's `user_id`
6. Returns JSON with success status and new `event_id`

Database Fields Created:

- `category`, `ename`, `description`, `date`, `time`, `venue`, `volunteers_needed`, `created_by`, `photo`

Response:

```
{
  "success": true,
  "message": "Event created successfully!",
  "event_id": 25
}
```

2. `add_notice.php` (`src/php/admin/add_notice.php`)

Purpose: Create System-Wide Announcements/Notices

Input Parameters:

- `subject` - Notice title
- `body` - Notice content/message

Process:

1. Stores notice in database with timestamp
2. May be displayed to all students on dashboard
3. Creates system-wide announcement

3. `update_event.php` (`src/php/admin/update_event.php`)

Purpose: Modify Existing Event Details

What it does:

- Updates any event fields: name, category, location, date, time, description, `volunteers_needed`, photo
- Handles photo replacement if new photo provided
- Used in event editing form

Input Parameters:

- `event_id` - Event to update
 - Any fields to be modified
-

4. `get_all_events.php` (`src/php/admin/get_all_events.php`)

Purpose: Fetch All Events with Metadata (Admin Dashboard)

What it does:

- Returns all events in database (no limit)
- Includes complaint count and feedback count for each event
- Sorted by date descending
- Returns JSON array for admin list display

Response Fields:

- All event fields
- `complaint_count` - Number of complaints for event
- `feedback_count` - Number of feedback submissions

Used by: Admin dashboard "All Events" list

5. `get_all_users.php` (`src/php/admin/get_all_users.php`)

Purpose: Fetch All Registered Users (Admin Dashboard)

What it does:

- Returns all users in system
- Includes all user details: name, email, phone, role, photo, address
- Used for user management list

Used by: Admin dashboard "Active Users" / "Edit Users" lists

6. `get_event_complaints.php` (`src/php/admin/get_event_complaints.php`)

Purpose: Fetch All Complaints for Specific Event

Input: `event_id`

What it does:

- Retrieves all complaints filed for an event
- Includes student name (filed by), complaint subject, text, and timestamps
- Used to display complaint list for admin review

Response Fields:

- `complaint_id`, `subject`, `complaint`, `created_at`, `user name`
-

7. `get_complaint_details.php` (`src/php/admin/get_complaint_details.php`)

Purpose: Fetch Single Complaint with Full Details

Input: `complaint_id`

What it does:

- Returns single complaint record
 - Includes: complaint text, student info, event details, admin reply if exists
 - Used for complaint detail view modal
-

8. `get_event_feedback.php` (`src/php/admin/get_event_feedback.php`)

Purpose: Fetch All Feedback for Specific Event

Input: `event_id`

What it does:

- Retrieves feedback submissions for event
 - Includes rating (1-5 stars) and feedback text
 - Shows student information
 - Used for feedback review list
-

9. `get_event_request.php` (`src/php/admin/get_event_request.php`)

Purpose: Fetch Single Student Event Request Details

Input: `request_id`

What it does:

- Retrieves student-submitted event request
 - Includes request details, status (pending/approved/rejected)
 - Used for detailed view of event request
-

10. `get_event_requests.php` (`src/php/admin/get_event_requests.php`)

Purpose: Fetch All Pending Event Requests (Admin Queue)

What it does:

- Returns all event requests from students
- Shows request status, proposed event details
- Sorted by date received (newest first)
- Used in admin dashboard event requests list

Response includes:

- `request_id`, student info, event details, status, `created_at`
-

11. `get_reported_items.php` (`src/php/admin/get_reported_items.php`)

Purpose: Fetch All Lost/Found Items (Not Yet Resolved)

What it does:

- Returns items reported as lost OR found
 - Filters out claimed/returned items
 - Shows item type (lost/found), name, description, location, photo
 - Used for admin item management
-

12. `get_reported_item_detail.php` (`src/php/admin/get_reported_item_detail.php`)

Purpose: Fetch Single Lost/Found Item Details

Input: Item ID

What it does:

- Returns complete item information
 - Includes: photo, description, location, reporter name, status
 - Used for item detail modal
-

13. `get_user_details.php` (`src/php/admin/get_user_details.php`)

Purpose: Fetch Complete User Profile Information

Input: `user_id`

What it does:

- Retrieves all user data: name, email, phone, address, role, photo
 - Used in user details modal in admin dashboard
-

14. `reply_complaint.php` (`src/php/admin/reply_complaint.php`)

Purpose: Store Admin Reply to Student Complaint

Input Parameters:

- `complaint_id` - Which complaint to reply to
- `reply_text` - Admin's response message

What it does:

1. Updates complaint record with admin reply
2. Sets `replied_by` (admin name), `replied_at` (timestamp)
3. Marks complaint as addressed
4. May trigger email to student with reply

15. conclude_event_request.php (src/php/admin/conclude_event_request.php)

Purpose: Approve or Reject Student Event Request

Input Parameters:

- request_id - Event request to process
- status - "approved" or "rejected"

What it does:

1. Updates event_requests table with decision
2. Updates status field
3. May trigger email notification to student
4. If approved, may create event from request details

16. update_item_status.php (src/php/admin/update_item_status.php)

Purpose: Mark Lost/Found Items as Resolved

Input Parameters:

- item_id - Item to update
- new_status - Status change (claimed/returned/resolved)

What it does:

1. Updates item status in database
2. May notify item reporter
3. Removes from active items list once resolved

17. email_helper.php (src/php/admin/email_helper.php)

Purpose: Email Utility Functions for Admin Operations

What it does:

- Contains helper functions for sending emails
- Used by: reply_complaint.php, conclude_event_request.php, etc.
- May include templates for email formatting
- Integrates with PHPMailer for SMTP email sending

Student PHP Files

Located in src/php/student/ - All require student role or authentication

1. register_event.php (src/php/student/register_event.php)

Purpose: Register Student for Event (Participant or Volunteer)

Input (JSON):

```
{
  "event_id": 5,
  "p_type": "participant" // or "volunteer"
}
```

Process:

1. Checks if user is logged in
2. Validates event exists
3. Prevents duplicate registrations
4. Inserts into participators table with status "pending"
5. Returns success/failure with 201 (created) or error HTTP code

Response on Success:

```
{
  "success": true,
  "message": "Successfully registered for event",
  "p_type": "participant"
}
```

2. submit_complaint.php (src/php/student/submit_complaint.php)

Purpose: File Complaint About Event

Input Parameters:

- event_id - Event complaint is about
- subject - Complaint title
- complaint - Detailed complaint text

What it does:

1. Stores complaint in database
2. Links to event and student
3. Stores timestamp
4. Sets initial status as "pending" (awaiting admin reply)
5. Returns confirmation

Database Record:

- complaint_id (auto), user_id, event_id, subject, complaint, created_at, reply (NULL initially)

3. submit_feedback.php (src/php/student/submit_feedback.php)

Purpose: Submit Event Feedback and Rating

Input Parameters:

- event_id - Event to rate
- rating - Rating 1-5
- feedback - Feedback text

What it does:

1. Creates feedback record
2. Stores rating (numeric 1-5)
3. Stores feedback comments
4. Links to event and student
5. Prevents duplicate feedback (one per event per student)

Used by: Post-event feedback modal

4. request_event.php (src/php/student/request_event.php)

Purpose: Student Proposes New Event (Admin Approval Queue)

Input Parameters:

- event_name - Proposed event name
- event_category - Category
- venue - Proposed venue
- date - Proposed date
- description - Event description
- volunteers_needed - Estimated volunteers
- photo - Event photo (optional)

Process:

1. Validates all required fields
2. Handles photo upload if provided
3. Creates event_requests record
4. Sets status to "pending" (waiting for admin approval)
5. Stores request with student info

Database Storage:

- request_id, user_id, event data fields, status, created_at

Next Step: Admin reviews in get_event_requests.php and decides via conclude_event_request.php

5. report_item.php (src/php/student/report_item.php)

Purpose: Report Lost or Found Item at Event

Input Parameters:

- item_name - Name of item
- item_description - What it looks like
- item_type - "lost" or "found"
- location - Where lost/found
- event_id - Associated event
- item_photo - Photo of item

Process:

1. Validates item type (must be "lost" or "found")
2. Handles photo upload to uploads/item_photos/
3. Stores item record with student as reporter
4. Creates item listing visible to other students
5. Sets status to "active" (not yet claimed/resolved)

Database Fields:

- id, user_id, event_id, item_name, item_description, item_type, location, photo, created_at, status
-

6. get_my_registrations.php (src/php/student/get_my_registrations.php)

Purpose: Fetch Student's Event Registrations (Upcoming)

What it does:

- Returns events student is registered for with future dates
- Shows participation type (participant/volunteer)
- Used for "My Events" or "Registered Events" list

Query Logic:

- FROM events JOIN participants
 - WHERE event.date >= today AND participants.user_id = current_student
 - Shows only upcoming events
-

7. get_my_event_requests.php (src/php/student/get_my_event_requests.php)

Purpose: Fetch Student's Event Requests with Status

What it does:

- Returns all events requested by current student
- Shows approval status: pending/approved/rejected
- Used in history modal for tracking request progress

Response includes:

- request_id, event details, status, submitted_date
-

8. get_my_complaints.php (src/php/student/get_my_complaints.php)

Purpose: Fetch Student's Filed Complaints

What it does:

- Returns all complaints submitted by student
- Shows associated event, complaint status
- Indicates if admin has replied
- Used in history modal

Response includes:

- complaint_id, subject, event info, status, reply (if any)
-

9. get_my_feedback.php (src/php/student/get_my_feedback.php)

Purpose: Fetch Student's Submitted Feedback

What it does:

- Returns all feedback submitted by student
- Shows event, rating, and feedback text

- Used for feedback history tracking
-

10. `get_admin_replies.php` (`src/php/student/get_admin_replies.php`)

Purpose: Fetch Admin Replies to Student Complaints

What it does:

- Returns only complaints that have admin replies
 - Filters for current student's complaints
 - Used in "Replies" section of student dashboard
-

11. `get_participated_events.php` (`src/php/student/get_participated_events.php`)

Purpose: Fetch Upcoming Events Student is Participating In

What it does:

- Returns future events where student is registered
 - Filters by date >= today
 - Used for "My Upcoming Events" list
-

12. `get_past_participated_events.php` (`src/php/student/get_past_participated_events.php`)

Purpose: Fetch Completed Events Student Participated In

What it does:

- Returns past events (date < today) where student was registered
 - Used for feedback submission (only past events can be reviewed)
 - Used in history of events attended
-

13. `get_reported_items.php` (`src/php/student/get_reported_items.php`)

Purpose: Fetch Items Reported by Current Student

What it does:

- Returns lost/found items reported by the current student
 - Shows report date, item type, status
 - Used in student's item history
-

14. `get_user_detail.php` (`src/php/student/get_user_detail.php`)

Purpose: Fetch Current Student's Own Profile Info

What it does:

- Returns logged-in student's full profile
 - Used for profile view/edit on student dashboard
-

Database Connection

`db_con.php` (`src/database/db_con.php`)

Purpose: Database Connection Initialization

What it does:

- Creates MySQLi connection to database
- Sets connection parameters (host, user, password, database)
- Used by all PHP files via `require_once`

Typical usage:

```
require_once __DIR__ . '/../../database/db_con.php';  
// Now $conn is available for queries
```

PHPMailer Integration

phpmailer/ directory (src/php/phpmailer/)

Purpose: Email Service Library

Contents:

- PHPMailer library files
- get_oauth_token.php - OAuth token generation for Gmail/OAuth2 authentication
- Configuration and SMTP setup

Used by:

- mail.php
- auto_mail.php
- email_helper.php

For sending emails with SMTP:

```
use PHPMailer\PHPMailer\PHPMailer;
$mail = new PHPMailer();
$mail->isSMTP();
// ... configuration
```

Data Flow Summary

1. User Registration Flow

```
signup.html → signup.php → users table → (auto-login) → student dashboard
```

2. Event Registration Flow

```
event_details_modal.html → register_event.php (AJAX) → participators table → success response
```

3. Admin Event Approval Flow

```
request_event.html → request_event.php → event_requests table
→ get_event_requests.php (admin views)
→ conclude_event_request.php (admin approves)
→ creates new event → auto_mail.php (notifies student)
```

4. Complaint and Reply Flow

```
complaint_modal.html → submit_complaint.php → complaints table
→ admin views via get_event_complaints.php
→ admin replies via reply_complaint.php
→ student sees reply via get_my_complaints.php
→ auto_mail.php notifies student
```

5. Event Discovery Flow

```
Homepage → fetch get_upcoming_events.php → display cards
→ fetch get_recent_events.php
→ fetch get_popular_events.php
```

Key Observations

🔍 What Works Well:

- Clear separation between public, admin, and student endpoints
- Consistent use of prepared statements (SQL injection prevention)
- JSON responses for AJAX operations
- File upload handling with validation
- Session-based authentication
- Logical grouping of functionality

⚠️ Areas for Improvement:

- No pagination - some endpoints return all records
- No rate limiting on public endpoints
- Mixed error handling patterns
- Some duplicate code across admin/student files
- No request logging or audit trail
- Could benefit from middleware/wrapper functions

Quick Reference: Which PHP to Use When

Need	PHP File
User login	login.php
Create account	signup.php
View event details	get_event_details.php
See upcoming events	get_upcoming_events.php
Register for event	register_event.php (student)
Create event	add_event.php (admin)
View all events	get_all_events.php (admin)
File complaint	submit_complaint.php (student)
Reply to complaint	reply_complaint.php (admin)
Request event creation	request_event.php (student)
Approve event request	conclude_event_request.php (admin)
Report lost item	report_item.php (student)
View lost items	get_reported_items.php (admin)